
Tutorial 6

Securing Web Applications

Computer Security

BCS2420

Part A

1. Which statement about the Same-Origin Policy (SOP) is **incorrect**?
 - ☐ SOP compares the scheme, host, and port to decide if two resources share the same origin.
 - ☐ SOP forbids a script from `https://bank.com` to directly access any data from `https://bank.com:8443`.
 - ☐ SOP can sometimes be relaxed by changing the `document.domain` property to a shared parent domain.
 - ☐ SOP does not apply if two pages share the same top-level domain (TLD, e.g., `.com`).
2. Which of the following best describes a drive-by download attack?
 - ☐ An attacker sends a phishing email with a malicious attachment that requires user execution.
 - ☐ The browser automatically fetches malicious scripts from a compromised website, exploiting vulnerabilities without explicit user action.
 - ☐ A user intentionally installs a Trojan horse from a third-party software repository.
 - ☐ The attacker physically installs malicious firmware on the victim's device.
3. Why are `HttpOnly` cookies often used to store session identifiers?
 - ☐ They reduce the size of HTTP requests by removing the domain field.
 - ☐ They ensure that the cookie is only accessible via

- client-side JavaScript.
- ☐ They protect cookies from direct access by client-side scripts, mitigating some XSS-based theft attempts.
 - ☐ They automatically encrypt the cookie's contents in transit.
4. Which of the following statements about Cross-Site Request Forgery (CSRF) is **true**?
- ☐ CSRF relies on malicious code injected into the victim's browser via script tags.
 - ☐ CSRF exploits the fact that browsers automatically attach cookies to outgoing requests for a domain.
 - ☐ CSRF is prevented by using a strictly long URL for requests.
 - ☐ CSRF cannot affect sites protected by HTTPS.
5. Which defense is best suited for preventing *stored* XSS attacks?
- ☐ Using an HTTPS connection for all pages
 - ☐ Stripping or sanitizing potentially dangerous HTML and script tags from user-generated content
 - ☐ Requiring a secret validation token in POST requests
 - ☐ Disallowing cookies completely
6. Which scenario exemplifies *Reflected* XSS?
- ☐ An attacker stores a malicious script in a site's comment section, and all visitors see the injected script.
 - ☐ An attacker uses a hidden IFRAME to submit a form to another origin.
 - ☐ A malicious link contains script code that the target server includes in its error message, immediately returning it to the user's browser.
 - ☐ A malicious script modifies the browser's DOM on a single-page application without server involvement.
7. What is the main purpose of *CSRF tokens*?
- ☐ To encrypt all request parameters in transit
 - ☐ To allow the attacker to reuse the user's session on a different machine
 - ☐ To verify that an HTTP request originated from the legitimate site's page/form rather than a third-party injection
 - ☐ To log out the user if suspicious behavior is

detected

8. Which *SQL injection* prevention method is recommended in modern web frameworks?
- ☐ Escaping all user input with backslashes before concatenation
 - ☐ Using parameterized (prepared) statements that keep SQL logic separate from user input
 - ☐ Disabling all forms that accept user input
 - ☐ Converting user input to uppercase before adding it to the query
9. The "Secure" cookie attribute is meant to:
- ☐ Restrict a cookie's usage to only the same origin domain
 - ☐ Ensure a cookie is only sent over HTTPS connections
 - ☐ Make the cookie accessible by client-side JavaScript
 - ☐ Invalidate the cookie if the user's IP address changes
10. Which approach helps mitigate *DOM-based XSS* vulnerabilities?
- ☐ Changing the default port to 8080 for web traffic
 - ☐ Sanitizing or encoding any untrusted data before adding it into the DOM or URL fragments
 - ☐ Disabling all cross-domain iframes
 - ☐ Using a multi-factor authentication token in each GET request
-

Part B

1. HTTP Cookies and Authentication

- **Question:** In web-based authentication, why are cookies so critical to maintaining sessions? Explain how malicious attackers might steal or abuse these cookies, and propose **two** different defensive measures (beyond `HttpOnly`) to protect them.

2. Mixed Content and HTTPS

- **Question:** A website loads its main page over HTTPS but includes some scripts and images over HTTP. Discuss why this is dangerous. In your answer, explain how an

active network attacker could exploit this setup to compromise the site's security.

3. **CSRF vs. XSS**

- **Question:** Compare and contrast Cross-Site Request Forgery (CSRF) with Cross-Site Scripting (XSS). How can an application be vulnerable to both simultaneously, and how do the mitigation strategies differ?

4. **Subdomain Scoping of Cookies**

- **Question:** How do the `Domain` and `Path` attributes of an HTTP cookie affect which subdomains (and URLs) can read the cookie? Provide an example scenario where misconfigured domain scoping leads to a security vulnerability.

5. **SQL Injection Attack Patterns**

- **Question:** What are **two** advanced SQL injection patterns (beyond the simple `' OR 1=1 --`)? For each pattern, briefly explain the exploitation idea and why naive sanitization might fail against it.

6. **Prepared Statements vs. Escaping**

- **Question:** In the context of mitigating SQL injection, why do **prepared statements** (parameterized queries) often outperform manual **escaping** or **blacklist**-based sanitization? Provide a short example in pseudo-code that illustrates how prepared statements prevent injection.

7. **CSP (Content Security Policy)**

- **Question:** Outline how a Content Security Policy can help mitigate XSS attacks. Mention at least **two** specific directives (e.g., `default-src`, `script-src`) and how they reduce the attack surface.

8. **Impact of XSS on SOP**

- **Question:** If an attacker successfully injects JavaScript into a legitimate domain (via stored XSS), how might this effectively *bypass* the browser's same-origin policy? Detail the potential consequences for user data stored or accessible under that domain.

9. **Browser Security vs. Server Security**

- **Question:** Discuss the interplay between client-side (browser-based) security mechanisms (e.g., same-origin policy, XSS filters) and server-side measures (e.g., input validation, authentication checks). Why is a

“defense in depth” approach necessary?

10. **Designing a Secure File-Upload**

- **Question:** A web application allows users to upload images. What steps should the server take to ensure malicious scripts or executables cannot be uploaded or executed? Provide at least **three** distinct measures.
-

Part C

1. **Estimating Cookie Expiration Risks**

- **Question:** A session cookie has a **Max-Age** of **4 hours**. Assume an attacker can guess or brute-force a random session token with a probability of success **1 in 2 million** per hour.
 - (a) What is the approximate probability the attacker succeeds within 4 hours if attacks are continuous?
 - (b) If the server decides to reduce Max-Age to **1 hour**, what does the success probability become?

2. **SQL Injection and Database Size**

- **Question:** An attacker performs a blind SQL injection on a user table containing ~1,000,000 rows. Each guess to confirm a single row's column value takes ~1 second via a timing-based technique.
 - (a) If the attacker tries to enumerate an entire column (e.g., username) with a maximum length of 10 ASCII characters, how many total seconds might be required in the worst case (assuming 95 printable characters)?
 - (b) Propose one method to reduce this enumeration time for the attacker.

3. **XSS Injection Surface** - **Question:** A forum restricts posts to **2,000** characters. An attacker attempts to insert malicious JavaScript using *inline event handlers* or `<script>` tags.

- (a) Suppose each inserted payload is ~100 characters. How many potential injection attempts can they embed in a single post if the forum does not sanitize it?

- (b) If the forum actively escapes all angle brackets `<` and `>`, how might an attacker still attempt XSS?

4. **Reflected XSS via URL Parameters**

- **Question:** A website echoes user input from a URL parameter into its error message (reflected XSS). If each request takes ~500ms to confirm whether the injection succeeded, how many attempts can an attacker perform in **1 minute** of testing on a single URL?

5. **Cookie Domain Scope**

- **Question:** A site sets the `Domain` attribute of its cookie to `.example.com`. The organization has subdomains `a.example.com` and `b.example.com`.
 - (a) Which subdomains can read the cookie?
 - (b) If there are 10 subdomains, what are the security implications if one subdomain is compromised?
-